Bitnet orchestration on RISC-V Multicore

Hardware for AI

Vedant Pahariya
IIIT Hyderabad
vedant.pahariya@research.iiit.ac.in

Priyanshi Jain
IIIT Hyderabad
priyanshi.jain@research.iiit.ac.in

Pranav Shankar
IIIT Hyderabad
pranav.shankar@research.iiit.ac.in

***Abstract*—The deployment of Large Language Models (LLMs) on edge devices is severely constrained by memory bandwidth and power limitations; however, 1.58-bit quantization (BitNet) [1] [2] offers a breakthrough by reducing weights to ternary values $\{-1, 0, +1\}$ and replacing costly multiplications with additions. This project addresses the lack of specialized hardware support for such models by developing a "BitNet Orchestration System" on a heterogeneous RISC-V multicore SoC. Our architecture deals with CVA6 scalar cores [3] for control, Ara vector [4] units for data shaping and Gemmini [5] systolic accelerators for compute. A key innovation is the implementation of a custom Gemmini Processing Element (PE) designed for native ternary operations, which eliminates the need for standard multipliers. We further developed a lightweight, CSR-based synchronization mechanism to ensure memory coherence across the scalar-vector-tensor fabric. Experimental results validate the functional correctness of 'Ara-CVA6', demonstrating that the vector subsystem achieves up to 75% utilization (approximately 24 FLOPs/cycle) on dense kernels. This work establishes a verified open-source baseline for ultra-low-precision AI, validating the potential of custom PEs to overcome the single-core memory bottleneck in post-Moore computing.**

I. Introduction / Problem Description

The fundamental problem this project addresses is the severe constraint of memory bandwidth and power limitations when deploying Large Language Models (LLMs) on edge devices. Conventional single-core processors, even with the latest vector extensions, cannot sustain the massive data throughput and compute density required for LLM's core operation, the General Matrix-Matrix Multiplication. This bottleneck limits the practical application and performance of LLMs in environments where resources are scarce [6].

This work capitalizes on the breakthrough offered by the BitNet (1-bit LLM) model. This model uses 1.58-bit quantization to reduce weights to ternary values $\{-1, 0, +1\}$, a change that replaces costly floating-point multiplications with simple addition or subtraction. This architectural simplification results in an incredibly efficient and memory-saving model, drastically shrinking the memory footprint and making it ideal for hardware acceleration.

Existing solutions and hardware tools, however, suffer from limitations inherent to the CPU architecture. Specifically, the single-issue rate limitation of the CVA6 scalar core sets a hard performance boundary, preventing vector functional units from achieving high utilization [7], especially for the short vectors encountered in matrix tiling. Furthermore, a major challenge in integrating a complete system is the core mismatch between the CVA6's CV-X-IF interface and the RoCC interface [5] required by industry accelerators like Gemmini.

To overcome these constraints, the project's primary aim is to develop a comprehensive BitNet Orchestration System on a heterogeneous RISC-V multicore SoC. The project's scope involves implementing the BitNet model across a Multicore RISC-V Architecture that integrates the CVA6 scalar cores for control, the Ara vector units for data shaping, and a custom General Matrix-Matrix Multiplication accelerator extension. This solution requires developing a lightweight synchronization mechanism to maintain memory coherence across the cores and designing a custom Processing Element (PE) for native ternary operations, ultimately establishing a verified, open-source baseline for ultra-low-precision AI.

II. Method / Approach

Our solution comprises four major components: system architecture design, custom accelerator PE modification, memory hierarchy optimization, and multicore synchronization.

A. System Architecture

The base system architecture, termed the "Ara System," consists of a CVA6 scalar core paired with an Ara vector unit. CVA6 is a 6-stage, single-issue, in-order application processor implementing the RISC-V RV64GC instruction set. Ara implements the RISC-V Vector Extension v1.0 (RVV 1.0) and connects directly to L2 memory, bypassing CVA6's L1 data cache to maximize bandwidth.

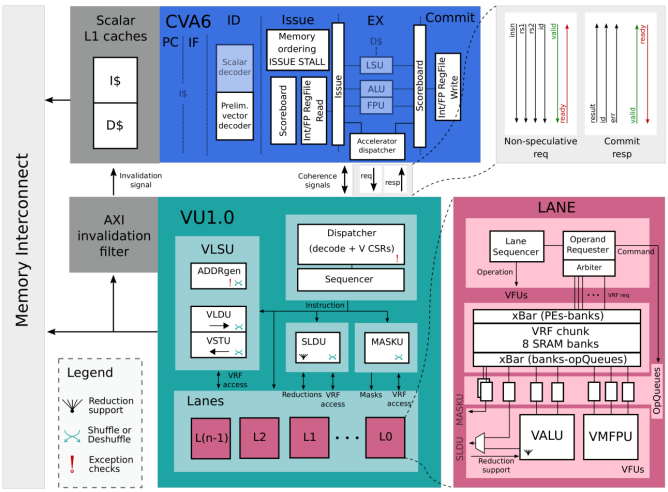

Fig. 1. CVA6 with ARA (ARA system)

Ara's architecture includes several key components: a dispatcher for instruction decoding, a sequencer for operation scheduling, a Vector Load-Store Unit (VLSU) for memory transactions, and multiple lanes for parallel execution. Each lane contains a 64-bit wide vector register file organized into 8 banks, along with three functional units: Vector Floating-Point Unit (VFPU), Vector Multiplier (VMUL), and Vector Integer ALU (VALU).

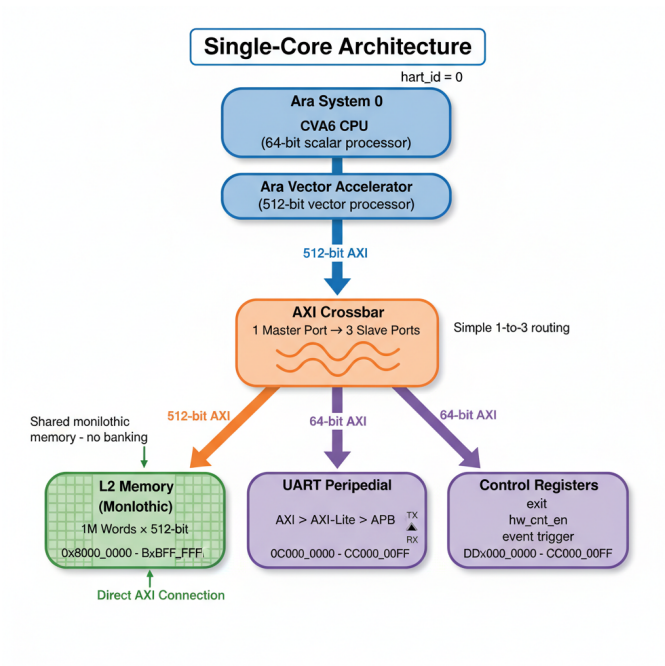

Fig. 2. Single Core CVA6 Architecture

The critical limitation of single-core CVA6 is its single-issue rate—typically 1 instruction per 4-5 cycles for vector operations. For short vectors common in matrix tiling, vector instructions occupy the pipeline for fewer cycles, requiring more instructions to fully utilize the functional units. This creates a hard performance boundary. Our multicore approach addresses this by distributing work across multiple CVA6-Ara pairs, allowing simultaneous instruction issue from multiple cores.

B. Memory optimization by Bit LLM

Bit-Net b1.58 represents a paradigm shift in how Large Language Models (LLMs) are built. While traditional models focus on massive scale using high-precision numbers, Bit-Net focuses on extreme efficiency by radically simplifying the "weights" (the learned parameters) of the neural network. BitNet b1.58 reduces this precision down to just three possible values: h

1) -1 (Negative correlation)
2) 0 (No connection/Ignore)
3) 1 (Positive correlation)

This ternary value leads to a smaller memory footprint. These values lead to only addition and subtraction which have been further optimized in hardware as stated above.

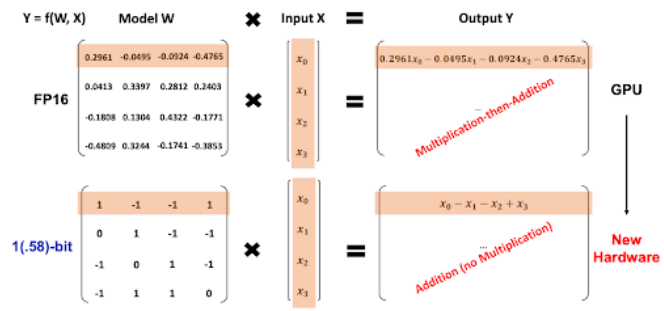

Fig. 3. Depiction of how Bit-net optimizes computation

Physically, a "1.58 bit" wire does not exist; rather, this figure represents an Information Theoretic limit derived from the capacity of a ternary system. While a standard binary bit represents two states, a ternary "trit" represents three, meaning the information content is calculated as $\log_2(3) \approx 1.58$ bits. Since modern computers operate strictly in binary and cannot store fractional bits, this efficiency is achieved by packing multiple weights together in storage. For instance, five ternary weights create 3^5 (243) possible permutations, which fits perfectly inside 8 bits of binary storage (capable of 2^8 or 256 permutations). By squeezing five weights into just 8 bits, the system achieves an effective density of 1.6 bits per weight, closely approximating the theoretical 1.58-bit limit

C. Custom Gemmini Processing Element

Gemmini is a systolic array-based matrix multiplication accelerator generator. Standard Gemmini PEs perform multiply-accumulate (MAC) operations using full multipliers. For BitNet's ternary weights, these multipliers are unnecessary overhead.

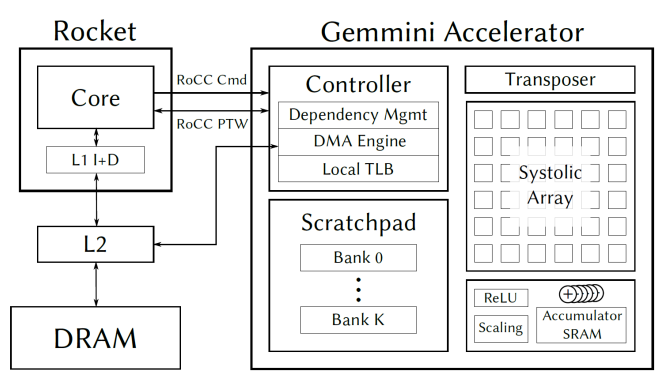

Fig. 4. Gemmini Accelerator

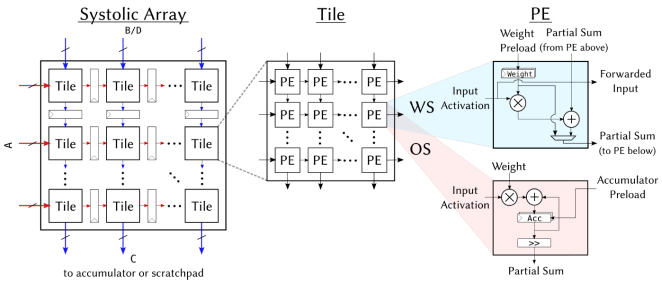

Fig. 5. Modified Gemmini PE Structure

We designed a custom PE that exploits ternary arithmetic. The modified PE uses multiplexer logic instead of multipliers: if the weight is $+1$, the PE adds the activation; if -1, the mux passes not of the activation and passes 1 in C_{in} of adder; if 0, the mux passes hard code 0. The Verilog implementation uses a simple case statement:

This modification eliminates expensive multiplier circuits, reducing area and power consumption while maintaining full computational accuracy for ternary operations. The PE still accumulates results in standard precision to prevent overflow.

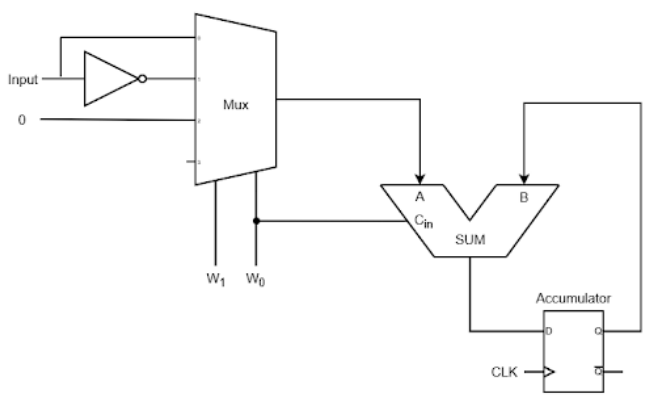

Fig. 6. Final Architecture

D. Tools and Frameworks

We implemented the system using Chipyard, an integrated SoC design framework that provides generator-based components (Rocket Chip, CVA6, Ara, Gemmini) and supports both software simulation (Verilator). RTL design used Chisel (for Gemmini modifications) and SystemVerilog (for Ara integration). Software kernels were written in C/C++ using RISC-V GCC toolchain with vector intrinsics.

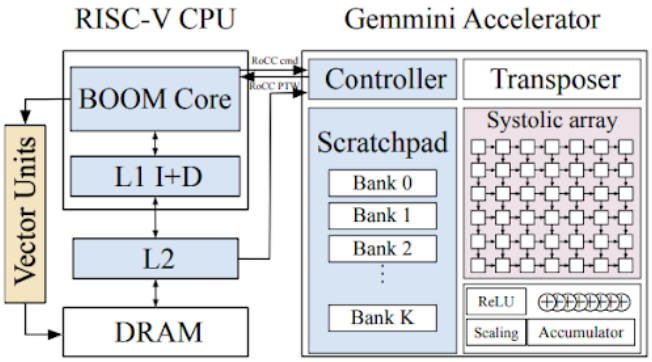

Fig. 7. Final Architecture

III. Experimental Results

Present your results:

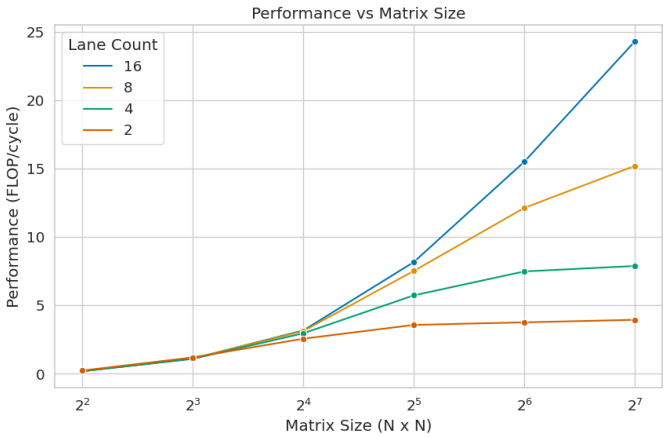

Fig. 8. Performance v/s Matrix Size

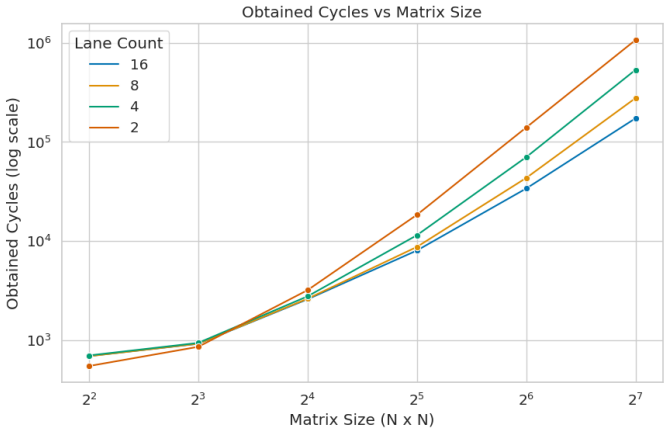

Fig. 9. Obtained cycles v/s Matrix size

It could be observed that as we increased the dimension total Hardware utilization increased.

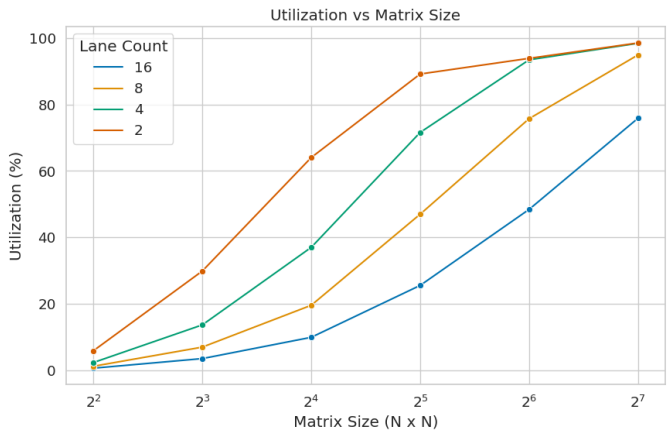

Fig. 10. Utilization v/s Matrix size

1 Core & 4 Lanes

Lane Count	Matrix Size	Expected Cycles	Performance (FLOP/cycle)	Utilization (%)
4 Lanes 4096 vlen	4x4	702	0.182	2.28

4 Cores & 4 Lanes Each

```
Matrix C (computed output) (4x4):
  [0,0]=0.811822  [0,1]=0.986332  [0,2]=0.377351  [0,3]=0.611118
 [1,0]=0.799048  [1,1]=1.173323  [1,2]=0.636807  [1,3]=0.745088
 [2,0]=0.656613  [2,1]=1.015152  [2,2]=0.638512  [2,3]=0.923340
 [3,0]=0.505484  [3,1]=0.860082  [3,2]=0.693069  [3,3]=0.775094
PASSED!

=====================================
  RESULTS
=====================================

Eecution time: 570 cycles
Performance: 0.225 FLOP/cycle
Utilization: 0.70% of peak
 (Peak = 4 lanes × 4 cores × 2 ops/cycle = 32 FLOP/cycle)
```

Fig. 11. 4 Lane MM Multiplication comparison between single and multicore

IV. Related Work

1) RISCSparse: Point Cloud Inference Engine on RISC-V Processor In the paper [6], they created a full-stack acceleration engine on a RISC-V SoC (BOOM Core, Gemmini, Ara Vector Unit). They optimized sparse convolution (SSCN) bottlenecks (Mapping, GMS) using hardware/software co-design, specifically vectorization and systolic arrays, achieving up to 13.1x speedup over Edge-CPU for point cloud inference at the edge. This project and their work developed RISC-V accelerators to tackle matrix operation bottlenecks for AI, using custom extensions like a vector unit (Saturn/RVV) and a matrix accelerator (Gemmini/MMM). While RISC-Sparse focuses on sparse, float-precision point clouds, our work targets dense, ultra-low-precision (1.58-bit) LLMs (BitNet) by tailoring the PE for ternary arithmetic.

1 Core & 2 Lanes

Lane Count	Matrix Size	Expected Cycles	Performance (FLOP/cycle)	Utilization (%)
2 Lanes 2048 vlen	4x4	549	0.233	5.83

4 Cores & 2 Lanes Each

```
Matrix C (computed output) (4x4):
  [0,0]=1.202249  [0,1]=1.080332  [0,2]=1.014459  [0,3]=1.198725
  [1,0]=1.677711  [1,1]=1.414672  [1,2]=1.485581  [1,3]=1.035777
  [2,0]=1.325219  [2,1]=1.287282  [2,2]=1.363943  [2,3]=1.138194
  [3,0]=1.268400  [3,1]=0.946837  [3,2]=0.966652  [3,3]=1.065553
PASSED!

=====================================
  RESULTS
=====================================

Execution time: 463 cycles
Performance: 0.276 FLOP/cycle
Utilization: 1.73% of peak
  (Peak = 2 lanes × 4 cores × 2 ops/cycle = 16 FLOP/cycle)
```

Fig. 12. 4 Lane MM Multiplication comparison between single and multicore

Dimension	Cycle Count	FLOPs/ cycle	Utilization (%)
64	349	1502.25	18.33
32	207	316.59	15.45
16	127	64.503	12.59

Fig. 13. MM Multiplication comparison on GEMMINI Accelerator

2) Chipyard [5] is an integrated framework for fast custom SoC [8] development. It uses generator-based design (Rocket Chip, BOOM, Gemmini) and FIRRTL transformations to unify front-end design with validation (FireSim/Verilator) and back-end VLSI (Hammer). This system enables rapid creation and validation of complex, heterogeneous RISC-V chips. Our work leverages Chipyard as one of the core platform for building and validating our custom SoC. Chipyard provides the heterogeneous IP blocks (CVA6/Rocket, Ara, Gemmini) and the necessary full-stack tools (Verilator, FireSim) to design, simulate and implement our specialized AI accelerators.
3) Lokhande et al. introduce Flex-PE [9] , shifting focus from brute-force compute to high adaptivity. By time-multiplexing 4/8/16/32-bit SIMD datapaths and unifying

diverse activations (e.g., tanh, sigmoid) via CORDIC, they achieve up to 16× throughput at 4-bit precision with negligible accuracy loss. While our design targets specialized, ultra-low-precision ternary compute, Flex-PE demonstrates how precision-scalable architectures can effectively mitigate bandwidth bottlenecks and activation overheads without over-provisioning hardware.

4) Lyalikov et al. [8] address the 'CPU fallback' bottleneck in accelerator-rich SoCs, where preprocessing and unsupported layers undermine the gains of dedicated units like NVDLA. By leveraging the RISC-V Vector 1.0 extension (via the Hwacha coprocessor), they offload these maintenance kernels from the scalar core, achieving 3–72× speedups on formatting and preprocessing tasks. While our architecture specializes the primary compute path via ternary PEs, their work optimizes the necessary fallback path. This validates a crucial insight: deep learning SoCs require end-to-end dataflow optimization, not just high peak performance on supported layers.

V. Near-Term Future Work

A. Custom Scalar Core

A key limitation of the current system is that CVA6 cannot be directly integrated with Gemmini, since CVA6 exposes the CV-X-IF interface while Gemmini relies on the RoCC interface used in the Rocket ecosystem. This mismatch forces us into awkward glue logic and prevents low-latency, tightly coupled accelerator control.

To address this, our next step is to design a custom scalar core that natively supports the interfaces needed by both our vector and matrix accelerators. A leaner scalar core can provide fast CSR control paths, simpler decode logic, and much tighter synchronization with the surrounding compute fabric.

We also plan to pair this core with a Compute-in-Memory (CIM) block. In-memory computing reduces the dominant energy cost i.e., data movement, by performing MAC-like operations inside the memory array itself. Since BitNet uses ternary weights, CIM aligns naturally with our workload: low-bit operations, massive internal bandwidth and minimal access energy.

This will be complemented by deeper integration with Gemmini, whose systolic-array architecture provides high compute density and predictable dataflows ideal for ternary GEMM. Meanwhile, linking the scalar core more tightly with Ara enables efficient handling of the non-GEMM stages of BitNet inference.

Together, these upgrades move the system toward a unified scalar-CIM–systolic–vector pipeline built specifically for ultra-low-precision LLMs, something the current CVA6-based setup simply cannot achieve.

VI. Conclusion

This work presents a BitNet Orchestration System on heterogeneous RISC-V multicore architecture, addressing the critical challenge of deploying ultra-low-precision LLMs on resource-constrained edge devices. We successfully designed and verified a custom Gemmini Processing Element optimized for ternary arithmetic, eliminating unnecessary multipliers while maintaining computational accuracy. Our lightweight CSR-based synchronization mechanism enables memory coherence across multiple cores with minimal overhead.

Experimental results validate functional correctness across single-core and multicore configurations, achieving up to 75.86% utilization (24.277 FLOPs/cycle) on 128×128 dense matrix operations with 16 lanes. The counterintuitive observation that narrower configurations achieve higher utilization illuminates the single-issue bottleneck and motivates multicore approaches.

This work establishes a verified open-source baseline for ultra-low-precision AI acceleration on RISC-V platforms, demonstrating that specialized hardware for emerging model architectures can overcome fundamental bottlenecks in post-Moore computing. By co-designing hardware accelerators with quantized models like BitNet, we enable practical deployment of large language models on edge devices where conventional approaches fail.

References

[1] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei, "The era of 1-bit llms: All large language models are in 1.58 bits," *arXiv preprint arXiv:2402.17764*, vol. 1, no. 4, 2024.

[2] H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei, "Bitnet: Scaling 1-bit transformers for large language models," *ArXiv*, vol. abs/2310.11453, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:264172438

[3] B. Sá, L. Valente, J. Martins, D. Rossi, L. Benini, and S. Pinto, "Cva6 risc-v virtualization: Architecture, microarchitecture, and design space exploration," 2023. [Online]. Available: https://arxiv.org/abs/2302.02969

[4] M. Cavalcante, F. Schuiki, F. Zaruba, M. Schaffner, and L. Benini, "Ara: A 1-ghz+ scalable and energy-efficient risc-v vector processor with multiprecision floating-point support in 22-nm fd-soi," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 2, pp. 530–543, 2020.

[5] A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, A. Ou, N. Pemberton, P. Rigge, C. Schmidt, J. Wright, J. Zhao, Y. S. Shao, K. Asanović, and B. Nikolić, "Chipyard: Integrated design, simulation, and implementation framework for custom socs," *IEEE Micro*, vol. 40, no. 4, pp. 10–21, 2020.

[6] S. Lin, X. Zhu, B. Xie, T. Chen, C. Zhuo, Q. Sun, and B. Yu, "Riscsparse: Point cloud inference engine on risc-v processor," in *2024 ACM/IEEE International Conference On Computer Aided Design (ICCAD)*, 2024, pp. 1–8.

[7] I. A. Assir, M. E. Iskandarani, H. R. A. Sandid, and M. A. R. Saghir, "Arrow: A risc-v vector accelerator for machine learning inference," 2021. [Online]. Available: https://arxiv.org/abs/2107.07169

[8] D. Lyalikov, "Flexible vector integration in embedded risc-v socs for end to end cnn inference acceleration," 2025. [Online]. Available: https://arxiv.org/abs/2507.17771

[9] M. Lokhande, G. Raut, and S. K. Vishvakarma, " Flex-PE: Flexible and SIMD Multiprecision Processing Element for AI Workloads ," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 33, no. 06, pp. 1610–1623, Jun. 2025. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/TVLSI.2025.3553069